# 태그 시스템 및 가비지 컬렉션 분석

## 개요

iroh-blob은 데이터 보호와 생명주기 관리를 위해 정교한 태그 시스템과 가비지 컬렉션 메커니즘을 제공합니다. 이 시스템은 두 가지 주요 태그 유형을 통해 데이터의 생존성을 관리하며, 참조 카운팅과 마크-앤-스위프 알고리즘을 결합한 효율적인 가비지 컬렉션을 구현합니다.

## 태그 시스템 아키텍처

### 태그 유형 분류

iroh-blob의 태그 시스템은 두 가지 주요 유형으로 구성됩니다:

**1. 임시 태그 (TempTag)**

```
#[must_use = "TempTag is a temporary tag that should be used to protect content

If you want to keep the content alive, use TempTag::leak()"]

pub  struct TempTag {

/// 보호하고 있는 해시와 포맷

inner: HashAndFormat,

/// 드롭 시 호출할 콜백 (옵션)

on_drop: Option<Weak<dyn TagDrop>>,

}
```

**특징:**

- **프로세스 생명주기 연동**: 프로세스가 실행되는 동안만 유효
- **자동 해제**: 스코프를 벗어나면 자동으로 해제됨
- **참조 카운팅**: 내부적으로 참조 카운터를 통해 생존성 추적
- **메모리 기반**: 메모리에만 존재하며 영구 저장되지 않음

**2. 영구 태그 (Permanent Tag)**

```
pub  struct Tag(pub Bytes);
```

**특징:**

- **영구 저장**: 데이터베이스에 영구적으로 저장됨
- **명시적 관리**: 사용자가 직접 생성/삭제해야 함
- **이름 기반**: 바이트 배열로 된 이름을 가짐
- **프로세스 독립적**: 프로세스 재시작 후에도 유지됨

## 태그 스코프 관리 시스템

### 스코프 기반 임시 태그 관리

```
pub(crate) struct TempTags {
scopes: HashMap<Scope, Arc<TempTagScope>>,
next_scope: u64,
}
```

```
pub(crate) struct TempTagScope(Mutex<TempCounterMap>);
```

**스코프 생명주기:**

1. **스코프 생성**: `create_scope()` 호출로 새로운 스코프 ID 할당
2. **태그 생성**: 특정 스코프 내에서 임시 태그 생성
3. **스코프 종료**: `end_scope()` 호출로 해당 스코프의 모든 태그 해제

**계층적 관리:**

```
impl TempTags {
pub fn create_scope(&mut self) -> (Scope, Arc<TempTagScope>) {
self.next_scope += 1;
let id = Scope(self.next_scope);
let scope = self.scopes.entry(id).or_default();
(id, scope.clone())
}
pub fn create(&mut self, scope: Scope, content: HashAndFormat) -> TempTag {
let scope = self.scopes.entry(scope).or_default();
scope.temp_tag(content)
}
}
```

# 데이터 보호 메커니즘

## 참조 카운팅 시스템

**포맷별 카운터 관리**

```rust
#[derive(Debug, Default, Clone)]
struct TempCounters {
/// Raw 포맷 임시 태그 수
raw: u64,
/// HashSeq 포맷 임시 태그 수
hash_seq: u64,
}
```

**카운터 동작 원리:**

- **증가**: 임시 태그 생성 시 해당 포맷의 카운터 증가
- **감소**: 임시 태그 드롭 시 해당 포맷의 카운터 감소
- **보호 판단**: 카운터가 0보다 크면 해당 데이터 보호됨

**컬렉션 보호 메커니즘**

```rust
/// 포맷에 따른 보호 범위:
/// - Raw: 해당 블롭만 보호
/// - HashSeq: 컬렉션과 모든 하위 블롭 보호
pub trait TagCounter: TagDrop + Sized {
fn on_create(&self, inner: &HashAndFormat);
fn temp_tag(self: &Arc<Self>, inner: HashAndFormat) -> TempTag;
}
```

### 보호된 데이터 관리

#### 저장소별 보호 메커니즘

```
// 메모리 저장소
protected: HashSet<Hash>,

// 파일 시스템 저장소
async fn handle_clear_protected(
cmd: ClearProtectedMsg,
protected: &mut HashSet<Hash>,
) -> ActorResult<()> {
protected.clear();
cmd.tx.send(Ok(())).await.ok();
Ok(())
}
```

**보호 상태 관리:**

- **추가**: 데이터 업데이트/설정 시 자동으로 보호 목록에 추가
- **확인**: 삭제 요청 시 보호 상태 확인
- **해제**: `clear_protected()` 호출로 일괄 해제

## 가비지 컬렉션 알고리즘

### 마크-앤-스위프 구현

### 1. 마크 단계 (Mark Phase)

```
async fn gc_mark_task(
store: &Store,
live: &mut HashSet<Hash>,
co: &Co<GcMarkEvent>,
) -> crate::api::Result<()> {
let mut roots = HashSet::new();
// 영구 태그 순회
let mut tags = store.tags().list().await?;
while let Some(tag) = tags.next().await {
let info = tag?;
roots.insert(info.hash_and_format());
}
// 임시 태그 순회
let mut tts = store.tags().list_temp_tags().await?;
while let Some(tt) = tts.next().await {
roots.insert(tt);
}
// 루트에서 시작하여 참조된 모든 해시 마킹
for HashAndFormat { hash, format } in roots {
if live.insert(hash) && !format.is_raw() {
let mut stream = store.export_bao(hash, ChunkRanges::all()).hashes();
```

```
while let Some(hash) = stream.next().await {
match hash {
Ok(hash) => { live.insert(hash); }
Err(e) => { /* 오류 처리 */ }
}
}
}
}
Ok(())
}
```

**마크 단계 특징:**

- **루트 수집**: 모든 영구 태그와 임시 태그를 루트로 수집
- **재귀 탐색**: HashSeq 포맷의 경우 하위 해시들도 재귀적으로 마킹
- **병렬 처리**: 비동기 스트림을 통한 효율적인 탐색

**2. 스위프 단계 (Sweep Phase)**

```
async fn gc_sweep_task(
store: &Store,
live: &HashSet<Hash>,
co: &Co<GcSweepEvent>,
) -> crate::api::Result<()> {
let mut blobs = store.blobs().list().stream().await?;
let mut count = 0;
```

```
let mut batch = Vec::new();
while let Some(hash) = blobs.next().await {
let hash = hash?;
if !live.contains(&hash) {
batch.push(hash);
count += 1;
}
// 배치 단위로 삭제 (100개씩)
if batch.len() >= 100 {
store.blobs().delete(batch.clone()).await?;
batch.clear();
}
}
if !batch.is_empty() {
store.blobs().delete(batch).await?;
}
store.sync_db().await?;
Ok(())
}
```

**스위프 단계 특징:**

- **배치 삭제**: 100개씩 묶어서 효율적으로 삭제
- **안전한 삭제**: 마크되지 않은 블롭만 삭제

- **동기화**: 삭제 완료 후 데이터베이스 동기화

## 가비지 컬렉션 설정

### GC 구성 옵션

```
#[derive(derive_more::Debug, Clone)]
pub struct GcConfig {
/// GC 실행 간격
pub interval: std::time::Duration,
/// 수동 보호 콜백 (옵션)
pub add_protected: Option<ProtectCb>,
}

pub type ProtectCb = Arc<
dyn for<'a> Fn(
&'a mut HashSet<Hash>,
) -> Pin<Box<dyn std::future::Future<Output = ProtectOutcome> + Send + Sync + '
+ Send + Sync + 'static,
>;
```

**설정 가능한 옵션:**

- **실행 간격**: 주기적 GC 실행 간격 설정
- **보호 콜백**: 사용자 정의 보호 로직 추가 가능
- **중단 제어**: 콜백에서 GC 실행 중단 가능

### GC 실행 프로세스

```
pub  async  fn  run_gc(store: Store, config: GcConfig) {
let  mut  live = HashSet::new();
loop {
live.clear();
tokio::time::sleep(config.interval).await;
// 사용자 정의 보호 콜백 실행
if  let Some(ref  cb) = config.add_protected {
match (cb)(&mut  live).await {
ProtectOutcome::Continue => {}
ProtectOutcome::Abort => {
info!("abort gc run: protect callback indicated abort");
continue;
}
}
}
if  let Err(e) = gc_run_once(&store, &mut  live).await {
error!("error during gc run: {e}");
break;
}
}
}
```

# 실제 사용 패턴

## 임시 태그 사용 예제

### 기본 사용법

```
// 데이터 추가 시 임시 태그 자동 생성
let temp_tag = store.add_bytes(b"hello world".to_vec()).temp_tag().await?;
let hash = *temp_tag.hash();

// 스코프 벗어나면 자동 해제
drop(temp_tag);
```

### 수동 임시 태그 생성

```
// 기존 해시에 대한 임시 태그 생성
let temp_tag = store.tags().temp_tag(hash_and_format).await?;

// 영구 보호로 전환 (leak)
temp_tag.leak(); // 프로세스 종료까지 보호
```

## 영구 태그 사용 예제

### 태그 생성 및 관리

```
// 영구 태그 생성
```

```
store.tags().set("my-important-data", hash_and_format).await?;


// 태그 조회
let tag_info = store.tags().get("my-important-data").await?;


// 태그 삭제
store.tags().delete("my-important-data").await?;
```

### 만료 태그 구현

```
async fn create_expiring_tag(
store: &Store,
hashes: &[Hash],
prefix: &str,
expiry: SystemTime,
) -> anyhow::Result<()> {
let expiry = chrono::DateTime::<chrono::Utc>::from(expiry);
let expiry = expiry.to_rfc3339_opts(chrono::SecondsFormat::Secs, true);
let tagname = format!("{prefix}-{expiry}");
if hashes.len() == 1 {
store.tags().set(&tagname, hashes[0]).await?;
} else {
let hs = hashes.iter().copied().collect::<HashSeq>();
```

```
store.blobs()
.add_bytes_with_opts(AddBytesOptions {
data: hs.into(),
format: BlobFormat::HashSeq,
})
.with_named_tag(&tagname)
.await?;
}
Ok(())
}
```

# 성능 최적화 기법

## 메모리 효율성

### 약한 참조 사용

```
pub struct TempTag {
inner: HashAndFormat,
// 강한 참조 대신 약한 참조 사용으로 순환 참조 방지
on_drop: Option<Weak<dyn TagDrop>>,
}
```

### 배치 처리

```
// GC에서 100개씩 배치 삭제
if batch.len() >= 100 {
store.blobs().delete(batch.clone()).await?;
batch.clear();
}
```

### I/O 최적화

#### 비동기 스트림 처리

```
// 블롭 목록을 스트림으로 처리하여 메모리 사용량 최소화
let mut blobs = store.blobs().list().stream().await?;
while let Some(hash) = blobs.next().await {
// 스트리밍 처리
}
```

#### 데이터베이스 동기화

```
// 삭제 완료 후 명시적 동기화
store.sync_db().await?;
```

## 오류 처리 및 복구

### GC 오류 처리

### 마크 단계 오류

```
match ev {
GcMarkEvent::CustomDebug(msg) => debug!("{}", msg),
GcMarkEvent::CustomWarning(msg, err) => warn!("{}: {:?}", msg, err),
GcMarkEvent::Error(err) => {
error!("error during gc mark: {:?}", err);
return Err(err);
}
}
```

### 스위프 단계 오류

```
// HashSeq 순회 중 오류 발생 시 경고 로그 출력 후 계속 진행
Err(e) => {
warn!("error while traversing hashseq: {e:?}");
}
```

## 태그 시스템 오류 처리

### 존재하지 않는 태그 감소

```
pub fn dec(&mut self, value: &HashAndFormat) {
let Some(counters) = self.0.get_mut(hash) else {
warn!("Decrementing non-existent temp tag");
```

```
return;
};
// 안전한 감소 (언더플로우 방지)
counters.dec(*format);
}
```

# 보안 고려사항

## 데이터 보호 보장

### 강제 삭제 방지

```
// 보호된 데이터는 강제 플래그 없이 삭제 불가
if !force && protected.contains(&hash) {
trace!("delete {hash}: skip (protected)");
continue;
}
```

### 태그 생성 권한

```
// 임시 태그는 스코프 기반으로 격리
pub fn create(&mut self, scope: Scope, content: HashAndFormat) -> TempTag {
let scope = self.scopes.entry(scope).or_default();
scope.temp_tag(content)
```

```
}
```

## 결론

iroh-blob의 태그 시스템과 가비지 컬렉션은 다음과 같은 핵심 특징을 제공합니다:

1. **이중 태그 시스템**: 임시 태그와 영구 태그의 조합으로 유연한 데이터 생명주기 관리
2. **스코프 기반 관리**: 계층적 스코프를 통한 체계적인 임시 태그 관리
3. **효율적인 GC**: 마크-앤-스위프 알고리즘과 배치 처리를 통한 성능 최적화
4. **안전한 보호**: 참조 카운팅과 보호 목록을 통한 데이터 무결성 보장
5. **확장 가능성**: 사용자 정의 보호 콜백을 통한 커스터마이징 지원

이러한 설계를 통해 iroh-blob은 대용량 데이터를 안전하고 효율적으로 관리할 수 있는 견고한 기반을 제공합니다.